

THE HEARTBEAT OF AUTONOMOUS INTELLIGENCE

The Agentic Loop and the Architecture That Connects
Models, Plugins, MCP, APIs, Tools and the World

“A model becomes operationally agentic not when it can think once, but when it can observe what happened, decide what to do next, act within authority, and know when to stop.”

ALEJANDRO REYNOSO

JUDGE BUSINESS SCHOOL.UNIVERSITY OF CAMBRIDGE.JULY 2026

Abstract

A language model can produce an answer, propose a plan, or request the use of a tool. None of those abilities, by itself, constitutes an autonomous system. Autonomy appears when the model is placed inside a controlled cycle that can preserve state, observe consequences, invoke capabilities, evaluate progress, and continue or stop according to explicit rules. This cycle is the *agentic loop*. It is the heartbeat of an autonomous model because it converts isolated inference into a path-dependent process of reasoning and action.

This paper develops a precise but pedagogical account of that loop. It distinguishes the logical minimum required for agency from the institutional minimum required for safe and auditable deployment. It explains the roles of the reasoning model, orchestrator, working state, memory, tool contracts, permissions, stopping conditions, error handling and human oversight. It then places plugins, the Model Context Protocol (MCP), APIs, skills and tools within the same architectural map. A plugin is treated as a package of reusable capabilities; MCP as a standardized discovery and invocation boundary; an API as a service-specific transaction interface; a skill as procedural knowledge; and a tool as an executable action exposed to the loop. These components are complementary, not interchangeable.

A running case—the morning inbox of the chief executive of a bank—shows how an agentic system can inspect messages, distinguish critical from non-critical matters, draft or send permitted replies, escalate consequential decisions, and prepare an auditable report. The case also makes visible the governance chassis beneath apparently effortless automation. The model proposes; the surrounding system authenticates, authorizes, validates, executes, records, and decides whether another cycle is justified. The result is a general architecture for autonomous work in which the visible answer is only the final page of an invisible, controlled organization.

Keywords: agentic loop; autonomous systems; agents; plugins; Model Context Protocol; MCP; APIs; tools; skills; orchestration; memory; governance; auditability; human oversight.

Contents

1	Introduction: When an Answer Is No Longer Enough	4
2	The Model Is Not the Autonomous System	4
3	The Anatomy of the Agentic Loop	5
3.1	Goal and trigger	5
3.2	Observation and context assembly	6
3.3	Reasoning and action selection	6
3.4	Policy and authorization gate	6
3.5	Execution	6
3.6	Observation of the result	6
3.7	State update	6
3.8	Termination, waiting or continuation	6
4	A Compact Formal Model	6
5	The Minimum Requirements	7
5.1	The logical minimum	7
5.2	The institutional minimum	8
6	Orchestration: The Loop Outside the Model	9
7	Skills and Tools: Knowing How and Being Able to Act	9

8 Plugins: The Packaged Capability	10
9 MCP: The Standardized Connector	10
9.1 What MCP does	11
9.2 What MCP does not do	11
10 APIs: The Underlying Transactions	11
11 Internal and External Resources	12
12 A Running Case: The Bank CEO's Morning Inbox	13
12.1 Cycle 1: establish scope	13
12.2 Cycle 2: inspect and classify	13
12.3 Cycle 3: propose actions	14
12.4 Cycle 4: execute and observe	14
12.5 Cycle 5: synthesize and stop	14
13 Memory, Context and Path Dependence	14
13.1 Working state	14
13.2 Conversation state	15
13.3 Durable memory	15
13.4 Audit state	15
14 Governance as the Chassis of Autonomy	15
14.1 Risk-tiered authority	16
14.2 Governance of the loop itself	16
15 Failure Modes: How Loops Go Wrong	16
15.1 The endless loop	17
15.2 Tool hallucination and schema mismatch	17
15.3 Authority creep	17
15.4 Stale or poisoned context	17
15.5 Duplicate side effects	17
15.6 False completion	17
15.7 Invisible handoff failure	17
16 From Prototype to Governed Production	17
17 Design Principles for the Agentic Age	18
18 Conclusion: The Heartbeat and the Chassis	18
A A Minimal Agentic Loop in Pseudocode	21
B Minimum-Requirements Checklist	21

1 Introduction: When an Answer Is No Longer Enough

THERE is a decisive difference between asking a model a question and delegating an objective to an autonomous system. In the first case, the model receives an input and produces an output. The interaction may be sophisticated, creative and even deeply reasoned, but its basic geometry remains linear: prompt, inference, response. In the second case, the system must remain alive after the first inference. It must inspect the world, determine whether information is missing, select an admissible action, observe the consequence of that action, revise its state and decide whether the objective has been achieved.

That second geometry is circular. It is a loop.

The distinction matters because contemporary discussion often attributes the properties of the whole autonomous system to the language model at its centre. We say that the model searched, remembered, opened a file, sent a message or monitored a condition. Strictly speaking, the model usually did none of those things directly. It emitted structured decisions inside a wider computational arrangement. Other components supplied the clock, credentials, network connection, file access, tool execution, durable memory, policy enforcement and evidence trail. The fluency of the interface hides the distribution of responsibility.

The **agentic loop** is the control structure that coordinates these responsibilities. It repeatedly asks a practical question: given the goal, the present state, the available capabilities and the result of the previous action, what should happen next? The answer may be to call a tool, request clarification, delegate to another agent, wait for an event, ask a human for approval, revise the plan, or declare completion. The loop converts reasoning from a single intellectual event into a sequence of bounded decisions.

This paper has four objectives. First, it explains the agentic loop with sufficient precision to distinguish it from ordinary prompting. Second, it identifies the minimum requirements for a system to qualify as operationally agentic. Third, it shows how plugins, MCP, APIs, tools, skills and internal or external resources fit together. Fourth, it adds the governance layer required when autonomy moves from classroom demonstration to institutional use.

The central proposition

The reasoning model is the engine. The agentic loop is the control system. Tools are the hands. Skills are the methods. Plugins package capabilities. MCP standardizes the connector. APIs often perform the underlying transactions. Memory preserves continuity. Governance determines what may happen. Audit records preserve evidence of what did happen.

2 The Model Is Not the Autonomous System

A language model maps an input context into a probabilistic output. That output can contain prose, code, a structured tool request, a plan or a final answer. The model can be exceptionally capable, but an isolated call remains an episode. Once the output is returned, nothing inherently forces another observation, another decision or another action.

An autonomous system adds a surrounding architecture. It gives the model a purpose, a state, a repertoire of actions and a mechanism for receiving feedback. It also establishes the conditions under which the process repeats. This is why agency should be understood as a *system property*, not merely as a model property.

In kernel terms the model is not the privileged component but the least trusted one: its output is untrusted input arriving at a system-call boundary, and the policy gate exists for exactly the reason that an operating system validates a syscall's arguments rather than trusting the process

that supplied them.

Three propositions follow.

1. **Reasoning is necessary but insufficient.** A model may reason brilliantly and still be unable to affect or inspect anything outside its input.
2. **Tool use is necessary for many tasks but insufficient for autonomy.** A single tool call followed by an answer is augmented generation; it becomes agentic only when results can change subsequent decisions inside a controlled cycle.
3. **Repetition alone is insufficient.** A script that calls a model ten times is not meaningfully autonomous if each call is blind to prior state, lacks a goal-directed decision rule, or cannot terminate intelligently.

“The loop is not merely repetition. It is repetition with memory, feedback, choice and a stopping rule.”

3 The Anatomy of the Agentic Loop

What follows is best read not as eight stages to be memorized but as a kernel to be derived: begin with a loop that observes, proposes, executes and updates, then admit each further mechanism only when a concrete failure makes its absence intolerable.

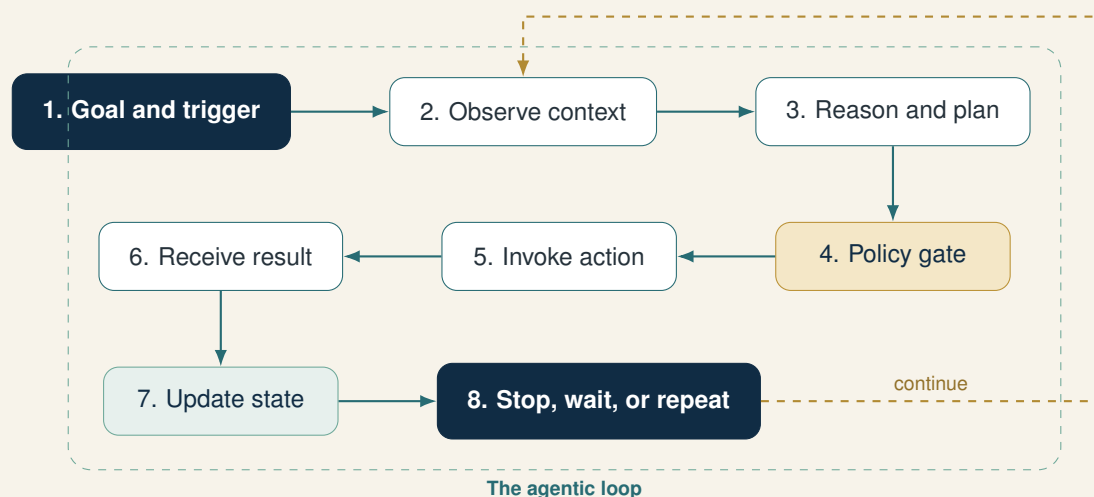


Figure 1: The agentic loop as a controlled cycle of observation, reasoning, action, feedback, and termination.

Figure 1 presents an eight-stage pedagogical representation. Implementations may combine or rename stages, but the underlying logic is remarkably stable.

3.1 Goal and trigger

Every loop begins with a reason to run. The trigger may be a user request, a scheduled time, a new email, a changed market price, a failed control, an incoming document or a signal from another agent. The goal defines the desired state: prepare the report, resolve the case, reconcile the account, answer the client or detect a material change.

A goal must be more than a slogan. “Review my inbox” is ambiguous. A deployable version specifies the relevant mailbox, time window, classification policy, actions that may be taken, matters that require escalation, completion criteria and the intended recipient of the report.

3.2 Observation and context assembly

The system assembles what the model is permitted to see: the goal, instructions, current working state, prior tool results, selected memory, available tool descriptions, policy constraints and budget. Context assembly is an engineering operation. It determines which facts are retrieved, which are summarized, which are omitted and which instructions have priority.

3.3 Reasoning and action selection

The model interprets the state and proposes the next move. In formal terms, it produces an action from a permitted action space. Possible actions include calling a tool, asking a question, delegating, revising the plan, returning a final result or waiting. The model may reason internally, produce a structured plan, or use explicit search procedures such as ReAct, reflection or tree search [Yao et al., 2023, Shinn et al., 2023, Zhou et al., 2024].

3.4 Policy and authorization gate

The proposed action is not automatically executed. The control system validates its schema, resolves the target, checks identity and permission, evaluates side effects and determines whether human approval is required. This is the architectural separation between *proposal* and *authority*.

3.5 Execution

An executor invokes a local function, hosted tool, plugin, MCP server, API, database operation, browser, application command or another agent. The executor handles timeouts, authentication, retries and errors. It should also protect against duplicated side effects when a failed call is retried.

3.6 Observation of the result

The result is normalized and returned to the loop. It may contain structured data, text, an error, a reference to a file, a confirmation that an action occurred, or an interruption requesting approval. Crucially, the result is not yet the answer. It is new evidence.

3.7 State update

The loop records what was attempted, what happened, what remains unresolved and what resources were consumed. Working memory is updated; durable memory may also be written. The new state becomes the input to the next decision.

3.8 Termination, waiting or continuation

The controller evaluates whether the objective is complete, impossible, unsafe, over budget or waiting on an external condition. It may return a final output, pause, escalate or begin another cycle. Current agent frameworks make this logic explicit: when a model returns a final output, the loop ends; when it emits a tool call or handoff, the result is appended and the model is called again [OpenAI, 2026c].

4 A Compact Formal Model

The loop can be expressed without pretending that every component is mathematically deterministic. Let the state at cycle t be

$$s_t = (g, x_t, m_t, \mathcal{T}_t, \rho, b_t),$$

where g is the goal, x_t is the current context, m_t is memory, $\mathcal{T}_t$ is the set of available tools, ρ is the policy and authority profile, and b_t is the remaining budget of time, money, tokens or turns.

The model proposes an action:

$$a_t \sim \pi_\theta(a \mid s_t),$$

with

$$a_t \in \{\text{tool call, handoff, question, wait, escalate, final answer}\}.$$

The policy layer transforms a proposal into an execution decision:

$$\hat{a}_t = G(a_t, \rho, s_t),$$

where G may approve, reject, modify, defer or route the action for human authorization. If execution is permitted, the environment returns an observation:

$$o_t = E(\hat{a}_t).$$

The controller then updates state:

$$s_{t+1} = U(s_t, \hat{a}_t, o_t),$$

and evaluates a stopping function:

$$\tau(s_{t+1}) \in \{\text{continue, complete, wait, escalate, fail safely}\}.$$

If the budget decreases strictly on every cycle and τ fires on exhaustion, the loop halts within b_0 cycles whatever the model proposes—a guarantee that no instruction, however carefully written, can supply.

This representation makes a crucial fact visible. The model policy π_θ is only one function in the system. Governance G , the environment E , state update U and termination τ are separate responsibilities. An organization that evaluates only model quality therefore evaluates only part of the autonomous system.

5 The Minimum Requirements

There are two useful meanings of “minimum.” The first is the *logical minimum* required for a recognizably agentic process. The second is the *institutional minimum* required before that process should be trusted with consequential work.

5.1 The logical minimum

A minimal agentic system requires six elements.

1. **A goal or objective.** The system must have a criterion against which progress can be evaluated.
2. **A decision mechanism.** Usually a language or reasoning model, although a planner, rules engine or hybrid may participate.
3. **At least one action channel.** The system must be able to do something other than produce unconstrained prose: call a tool, query data, modify state, communicate or delegate.
4. **Feedback.** The consequence of the action must be observable and available to a subsequent decision.
5. **State.** The next cycle must know something about the goal, prior actions, observations and unresolved work.

6. **A continuation and stopping rule.** The controller must know when another cycle is allowed and when the run must end.

Remove any one of these and the architecture loses an essential characteristic. Without a goal, it is unguided activity. Without a decision mechanism, it is a fixed workflow. Without action, it is reflection. Without feedback, it is open-loop execution. Without state, it cannot learn from its path. Without a stopping rule, it is an uncontrolled process.

5.2 The institutional minimum

The logical minimum is not a safe deployment standard. A bank, hospital, public authority or listed company needs at least five additional elements:

1. **Identity and authorization:** who or what is acting, for which user, on which resources and under what role.
2. **Input, output and target validation:** schemas, identifiers, data classification, constraints and error handling.
3. **Human oversight and escalation:** explicit thresholds for approval, interruption and contestability.
4. **Observability and auditability:** a trace of decisions, tool calls, results, policy gates, state transitions and artifacts.
5. **Operational limits:** budgets, timeouts, retry limits, idempotency protections, rollback or recovery plans, and a safe failure state.

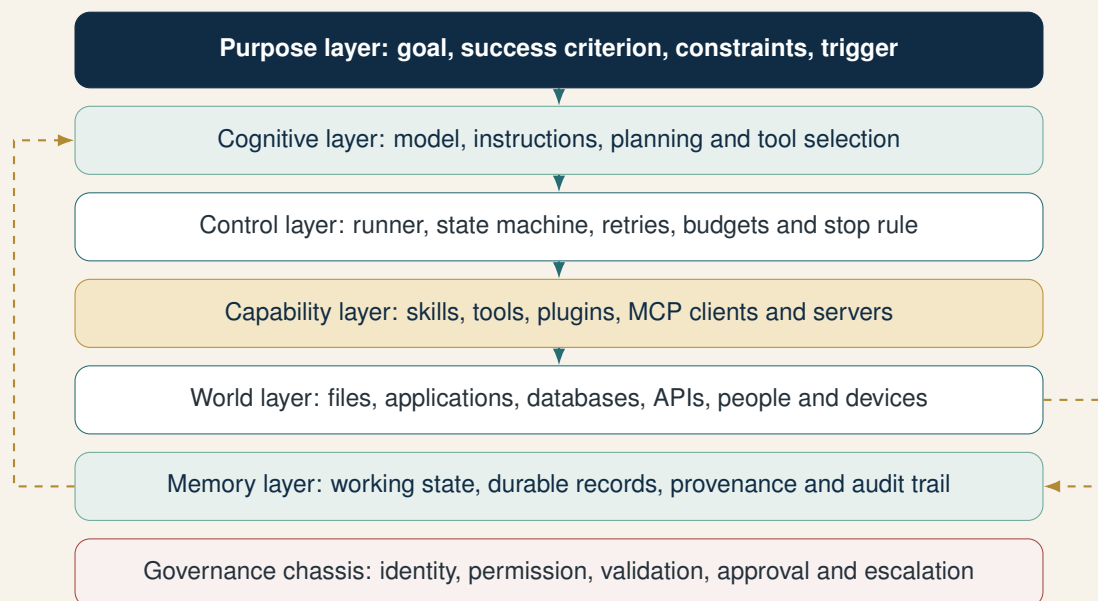


Figure 2: A deployable autonomous system is a layered architecture, not a single model.

The minimum is a ladder, not a single threshold

A classroom prototype may demonstrate agency with a goal, model, tool, feedback, state and stop rule. An institutional autonomous system must add authority, validation, observability, recovery and human control. Calling both systems “agents” is acceptable only if the difference in assurance is made explicit.

6 Orchestration: The Loop Outside the Model

The **orchestrator** or **runner** owns the lifecycle of the loop. It calls the model, parses its output, dispatches tools, captures results, updates state and decides whether to call the model again. It may also manage concurrency, delegation, rate limits, failures and long-running pauses.

This component is often invisible because agent frameworks provide it automatically. Yet it is the architectural source of persistence. A model cannot spontaneously wake itself tomorrow morning. A scheduler, event bus, automation service or external process must create the next run. Likewise, a model cannot reliably enforce its own maximum number of turns; the runner must count and stop them.

Four loop patterns are especially useful.

Pattern	Trigger	Typical behavior
Turn-based	A human message	The agent reasons, uses tools and returns one conversational result.
Goal-based	An objective and success criterion	The loop continues until the objective is satisfied, blocked or over budget.
Time-based	A schedule or deadline	The system wakes at defined times, observes changes and produces or updates artifacts.
Event-based	A condition or incoming signal	The loop starts when a price, email, file, alert or workflow state changes.
Proactive multi-agent	A persistent mission distributed among roles	Specialized agents research, execute, review and synthesize under an orchestrator.

These patterns can be combined. A time-based run may become goal-based once started. A goal-based loop may pause for human approval and resume from serialized state. A multi-agent system may use handoffs or expose specialist agents as tools. The conceptual invariant is the same: each new action is chosen in light of an updated state.

7 Skills and Tools: Knowing How and Being Able to Act

The distinction between skills and tools is central to the wider architecture.

A **skill** is reusable procedural knowledge. It describes how to perform a class of tasks: how to triage an inbox, review a contract, build a presentation, validate a PDF or analyze a portfolio. A good skill contains method, sequencing, constraints, quality standards, decision rules and references. It shapes the agent's behavior but does not necessarily grant access to any external system.

A **tool** is an executable capability. It reads a file, queries a database, calls an API, sends a message, runs code, changes a record or invokes another agent. Tools have operational consequences and therefore require contracts: names, descriptions, inputs, outputs, permissions, side effects and failure behavior.

	Skill	Tool
Primary function	Provides a method for doing work	Performs a concrete read, computation or action
Typical form	Instructions, templates, checks, examples	Function, command, hosted service, MCP tool or agent
Question answered	“How should this task be performed?”	“What can the system execute now?”
Main governance issue	Ownership, versioning, tests and permitted agents	Authorization, validation, side effects and audit
Example	“Triage bank email into urgent, reply-soon, waiting and FYI”	‘search_email’, ‘read_thread’, ‘draft_reply’, ‘send_email’

The two work together. A skill may instruct the agent to search before reading, read before drafting, verify the recipient before sending, and require approval for a sensitive reply. The tools make those steps executable.

8 Plugins: The Packaged Capability

The term **plugin** is sometimes used loosely to mean any integration. A more precise contemporary meaning is an installable package that extends an agentic environment. It may contain skills, reusable resources, assets, configuration, user-interface components and an MCP server. A plugin can therefore change how the system works, what it knows how to do and which external capabilities it can reach.

A plugin is not a protocol. It is a packaging and distribution unit.

This distinction explains why a plugin may be *skills-only*. If the required workflow operates entirely on information already available to the model, instructions and resources may be enough. When the plugin must read live data, authenticate a user, take action or connect to another service, it can include an MCP server [OpenAI, 2026b].

The plugin answers an organizational question: *what coherent extension are we installing?* The MCP server answers an interface question: *how are live tools and resources described, discovered and invoked?* An API answers a service question: *how does software transact with the underlying application or provider?*

9 MCP: The Standardized Connector

The Model Context Protocol is an open specification for connecting AI clients to tools and contextual resources. Its architecture separates a **host**, one or more **clients**, and one or more **servers**. The host coordinates the AI application and enforces permissions; each client maintains an isolated connection to a particular server; servers expose focused capabilities such as tools, resources and prompts [Model Context Protocol, 2025].

The protocol is valuable because it replaces many bespoke integrations with a machine-readable and discoverable contract. A client can request a tool list. Each tool can describe its name, purpose and JSON input schema. The model can then select a tool and produce arguments that conform to that schema. The server validates the request, performs the operation and returns structured or textual content. The result re-enters the agentic loop as a new observation [Model Context Protocol, 2026].

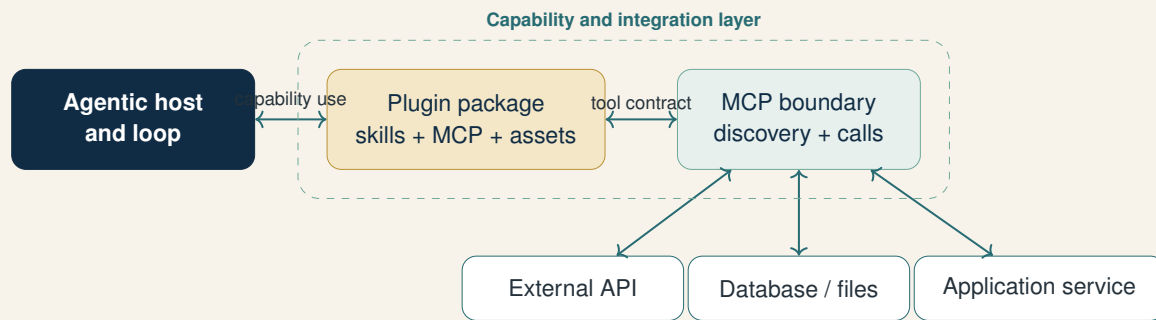


Figure 3: Plugin, MCP and API are complementary layers. An MCP server often calls an API, but neither is the agentic loop.

9.1 What MCP does

- standardizes discovery of capabilities;
- provides structured tool invocation and results;
- separates the AI host from service-specific implementations;
- supports local or remote servers;
- provides capability negotiation, lifecycle and authorization patterns;
- allows multiple focused servers to be composed while preserving boundaries.

9.2 What MCP does not do

MCP is not the language model, agent, planner, scheduler, database, memory system or governance regime. It does not create the objective, decide how long the system should run or prove that a tool's business logic is correct. It standardizes a boundary. The host and surrounding institution remain responsible for orchestration, trust and control.

The marketplace metaphor

Imagine the model entering a carefully regulated marketplace. Each shop displays a machine-readable catalogue: what it offers, which inputs are required, what may be returned and what permissions apply. MCP organizes the catalogues and the transaction language. The model may choose a shop, but the host controls which streets are open, which purchases require authorization and which receipts must be preserved.

10 APIs: The Underlying Transactions

An Application Programming Interface is a contract through which one software component requests data or operations from another. APIs predate modern agentic systems and remain foundational. A market-data API may return prices. An email API may search messages or create drafts. A banking API may retrieve transactions. A database API may execute a query.

An agent can use an API in at least three ways.

1. **Direct tool wrapper.** A developer writes a function that calls the API and exposes that function to the model as a tool.
2. **MCP server.** The MCP server exposes model-friendly tools and internally calls one or more APIs, libraries or databases.
3. **Hosted or application-native integration.** The platform supplies an already managed tool whose implementation uses APIs behind the scenes.

The second pattern is especially instructive. Suppose a raw enterprise API exposes 150 low-level endpoints and requires internal identifiers. Mirroring all 150 endpoints as agent tools would create a confusing and unsafe surface. A well-designed MCP server instead exposes a smaller set of user-goal-oriented tools: ‘find_client’, ‘get_exposure’, ‘prepare_brief’ and ‘request_approval’. It translates human intent into stable service operations, validates identifiers, removes secrets from outputs and separates reads from writes. Current plugin guidance explicitly recommends designing tools around user outcomes rather than mechanically copying an internal API [OpenAI, 2026d].

Layer	Primary purpose	Typical contents	Not equivalent to
Plugin	Package and distribute an extension	Skills, assets, MCP server, UI, metadata	The agentic loop
MCP	Standardize AI-to-capability discovery and calls	Tools, resources, prompts, schemas, protocol messages	The underlying business service
API	Expose a service-specific software interface	Endpoints, methods, authentication, responses	A model-aware tool catalogue
Tool	Give the loop one callable capability	Name, description, schema, executor	A complete workflow
Skill	Supply reusable procedural knowledge	Instructions, sequencing, tests, templates	Execution authority

11 Internal and External Resources

The loop may interact with resources on either side of an organizational perimeter.

Internal resources include local files, a private knowledge vault, enterprise databases, proprietary applications, internal services, code runtimes and devices. They may be reached through local functions, operating-system tools, internal APIs or local/remote MCP servers. “Internal” does not mean low risk. A payroll file or risk database may be more sensitive than a public website.

External resources include public web pages, market-data vendors, SaaS platforms, third-party databases, cloud services and counterparties. They are commonly reached through remote APIs, connectors, plugins, remote MCP servers or controlled browser/computer interfaces.

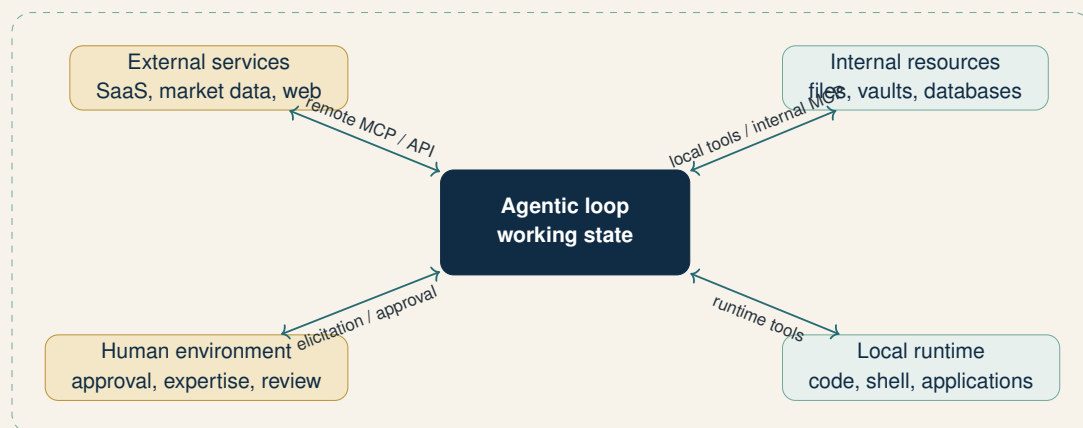


Figure 4: The loop can interact with both internal and external environments, but each boundary requires an explicit contract and authority model.

The important architectural question is not merely where the resource is located. It is how the trust boundary is crossed. Each connection should specify:

- identity and authentication;
- permitted accounts, datasets, directories or operations;
- input and output schemas;
- confidentiality and retention rules;
- read-only versus state-changing behavior;
- reversibility, confirmation and escalation requirements;
- failure, timeout and retry behavior;
- evidence to be logged.

This is where the automobile metaphor becomes useful again. The engine does not touch the road directly. The transmission, steering and braking systems mediate power. Similarly, the model should not touch an enterprise system through an unrestricted credential. A connector layer translates, limits and records the interaction.

12 A Running Case: The Bank CEO's Morning Inbox

Consider a chief executive who asks:

Every morning, review my inbox. Identify critical and non-critical messages. Handle the routine matters that policy permits, and prepare a concise report on the critical items that require my attention.

The request sounds simple. The loop reveals its hidden organization.

12.1 Cycle 1: establish scope

At 6:30 a.m., a scheduler starts the run. The orchestrator loads the CEO's instruction, the bank's email-handling skill, the current policy version, the approved mailbox identity and the previous morning's unresolved items. It discovers email tools from an installed plugin. The plugin may include the triage skill and an MCP server that exposes 'search_mail', 'read_thread', 'create_draft', 'send_reply' and 'label_thread'. The MCP server may, in turn, call the email provider's API.

The model does not receive unrestricted mailbox access merely because the plugin is present. The host supplies a scoped credential and a time-bounded query. The first tool call is a read:

```
search_mail(after=previous_run, mailbox=CEO, limit=200).
```

The returned message identifiers become observations. The loop updates its state: 47 new messages, 12 likely relevant, 35 low-salience or automated.

12.2 Cycle 2: inspect and classify

The model requests the 12 relevant threads. The server validates that the identifiers belong to the approved mailbox, retrieves them and returns normalized content. The skill instructs the model to classify along several dimensions: urgency, financial materiality, regulatory sensitivity, reputational risk, decision ownership, reply requirement and confidence.

A message about a regulatory inquiry is critical even if its tone is calm. A message from a board member may be critical because of governance context. A routine request to confirm attendance may be non-critical but still require a reply. The classification therefore uses institutional criteria, not merely sentiment.

12.3 Cycle 3: propose actions

The model proposes:

- draft acknowledgements for three routine messages;
- send two pre-authorized confirmations whose wording is fixed by policy;
- leave four informational messages untouched;
- escalate three messages: a regulatory inquiry, a liquidity issue and a personnel matter.

The proposal is routed through the governance gates. Sending a fixed-format confirmation may be allowed automatically. A free-form reply concerning a client, employee or regulator requires approval. The model can draft it, but cannot send it.

12.4 Cycle 4: execute and observe

The executor sends only the two permitted confirmations, using idempotency keys to prevent duplicate messages if the process is retried. It creates drafts for the other routine items and returns stable draft identifiers. The audit layer records the policy that permitted each action, the model's proposed arguments, the validated recipient, the final text and the provider's confirmation.

12.5 Cycle 5: synthesize and stop

The loop verifies that every relevant message is accounted for. It produces the CEO report:

1. three critical matters, ranked by consequence and deadline;
2. five matters already handled or prepared;
3. four informational messages with no action required;
4. uncertainties and links to source threads;
5. an explicit statement of what was sent, what remains a draft and what requires approval.

The run terminates because the inbox window has been exhausted, every selected thread has a disposition, the report has been generated and no unresolved tool call remains. The next morning requires a new scheduled run, not an immortal model process.

What actually happened

The email application stored the messages. Its API performed the underlying transactions. The MCP server exposed a focused, model-readable tool surface. The plugin packaged the skill and connection. The model classified and proposed. The orchestrator repeated the cycle. Policy gates controlled authority. Memory preserved unresolved items. The audit layer preserved evidence. The apparent "autonomous assistant" was a coordinated system.

13 Memory, Context and Path Dependence

An agentic loop requires state, but not all state is memory and not all memory belongs in the model context.

13.1 Working state

Working state contains what the current run needs: the goal, plan, completed steps, tool results, errors, budgets and unresolved questions. It may exist in an in-memory object, serialized run state or workflow database. It should be compact enough to support the next decision and precise enough to resume after interruption.

13.2 Conversation state

Conversation history preserves the sequence of messages and tool interactions within or across turns. It supports coherence, but unbounded transcript accumulation is expensive and can bury important instructions. Systems therefore summarize, select or normalize history.

13.3 Durable memory

Durable memory persists beyond the run: prior reports, approved decisions, user preferences, institutional policies, embeddings, knowledge graphs, vault notes or structured records. Retrieval should be selective and provenance-aware. A model should not treat an old summary as current policy merely because it is semantically similar.

13.4 Audit state

Audit records are not primarily prompts for future reasoning. They are evidence: timestamps, identities, model and skill versions, tool calls, approvals, results, hashes, errors and artifacts. Working memory may be edited or summarized; audit evidence should be append-only or otherwise protected from silent alteration.

The distinction matters because a loop is path-dependent. The meaning of the current state depends on the sequence that produced it. A market conclusion based on three prior observations differs from the same conclusion based only on the closing value. An inbox disposition after two failed send attempts differs from one produced without errors. Autonomy without path-aware state is fragile because the system cannot reliably explain or recover from its own history.

14 Governance as the Chassis of Autonomy

Greater autonomy expands the distance between human instruction and machine action. Governance must fill that distance.

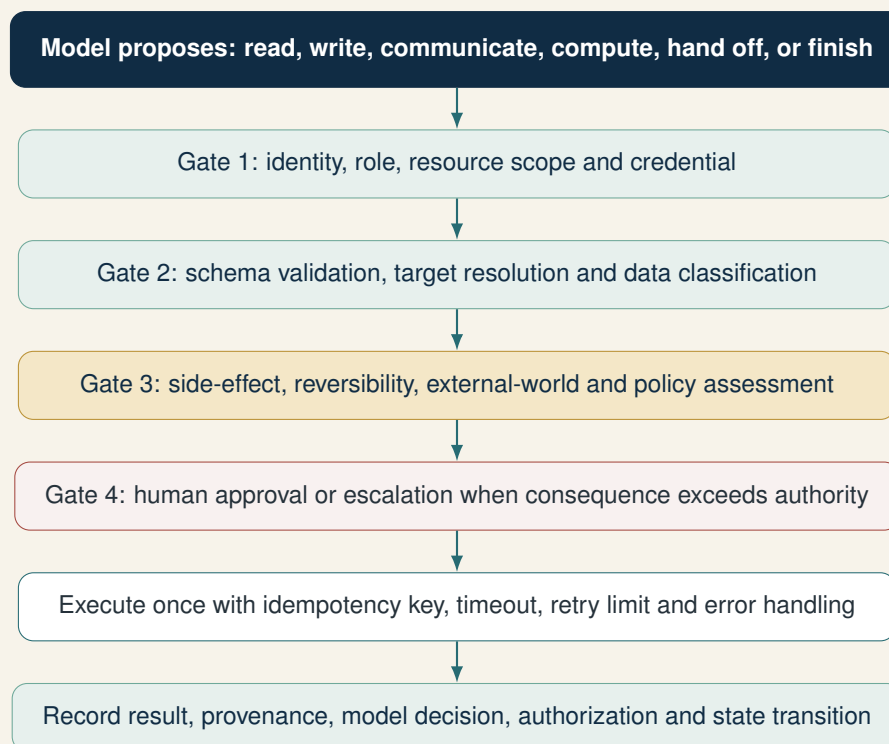


Figure 5: A governance-first execution path. The model proposes; the surrounding system decides what may actually happen.

The core principle is simple: *the model proposes; the surrounding system authorizes and*

executes. This separation reduces the temptation to encode critical controls only in natural-language instructions. Prompts matter, but institutional controls should also exist in deterministic code, permissions, schemas and workflow gates.

14.1 Risk-tiered authority

A useful authority model distinguishes four levels.

Tier	Typical action	Authority	Evidence
Observe	Search, read, compare, calculate	Automatic within scoped read access	Query, source, timestamp, result
Prepare	Draft, recommend, simulate, assemble	Automatic if no external state changes	Draft artifact, assumptions, confidence
Act	Send, publish, update, submit, execute	Policy-dependent; often requires confirmation	Resolved target, approval, provider receipt
Commit	Transfer value, sign, delete, disclose, bind institution	Explicit human or multi-party authorization	Full decision package and immutable audit record

An approval does not bypass the gate; it enters the gate as evidence, and the authorized action is revalidated against schema, scope and policy at the moment it executes—which may be hours after the human consented, under conditions that have since changed.

Human-in-the-loop should not mean that a person clicks “approve” on everything. That design produces fatigue and ceremonial control. Approval should be concentrated where consequence, ambiguity, irreversibility or authority justify it. Modern agent runtimes support pause, serialize, approve or reject, and resume patterns for sensitive tool calls [OpenAI, 2026a].

14.2 Governance of the loop itself

The institution should govern not only individual tools but also loop dynamics:

- maximum turns, runtime and cost;
- permitted recursion and delegation depth;
- conditions for retry;
- rules for parallel actions;
- model, skill and tool versions;
- memory write and retrieval policies;
- conditions for autonomous continuation;
- safe state when the process is uncertain or interrupted.

This approach aligns with the broader logic of the NIST AI Risk Management Framework: governance, mapping, measurement and management must accompany deployment, especially when generative systems introduce novel risks [National Institute of Standards and Technology, 2023, 2024].

15 Failure Modes: How Loops Go Wrong

The loop amplifies both competence and error. A poor answer may inconvenience a user once; a poor action policy may repeat, propagate and alter the environment.

15.1 The endless loop

The model repeatedly searches, revises or reflects without improving the state. The remedy is not merely a stronger prompt. The controller needs turn limits, progress measures, diminishing-return rules and escalation.

15.2 Tool hallucination and schema mismatch

The model requests a nonexistent tool or malformed arguments. Discovery, constrained schemas and deterministic validation reduce this risk. A failed call must become a structured observation, not a silent approximation.

15.3 Authority creep

An agent begins with permission to prepare and gradually behaves as if it may act. Read, draft, send, publish and delete should be separate capabilities with separate permissions. Tool descriptions and server-side enforcement must agree.

15.4 Stale or poisoned context

Old memory, malicious documents or external content can misdirect the model. Retrieval needs provenance and trust labels. Prompt injection is therefore not a reasoning failure to be corrected by better instructions but a context-assembly failure to be corrected in the memory manager: external text must reach the context window already labelled as data, with its provenance attached and its authority denied. Sensitive tools should remain protected even when the model is persuaded to request them.

15.5 Duplicate side effects

A timeout occurs after an external service completed an action but before confirmation returned. A naive retry sends the email or payment twice. Idempotency keys, provider receipts and reconciliation are therefore part of agent design.

15.6 False completion

The model produces a polished narrative even though one source failed or one required step was skipped. Completion should be tested against machine-readable criteria: every required item has a disposition, all mandatory tools succeeded or were explicitly waived, and unresolved uncertainties are surfaced.

15.7 Invisible handoff failure

One agent delegates to another, but context, authority or responsibility is lost. Handoffs require a defined task, relevant evidence, permitted tools and a return contract. The audit trail must preserve both the boundary and the final owner.

The stop rule is a safety control

The ability to continue is the essence of the agentic loop; the ability to stop is the essence of governed autonomy. Every production loop needs explicit outcomes for completion, waiting, escalation, budget exhaustion, policy rejection and unrecoverable error.

16 From Prototype to Governed Production

A sensible development path increases autonomy only as evidence increases.

1. **Stage 0—Single response.** The model explains what it would do. No tools or external state.

2. **Stage 1—Read-only augmentation.** The model can retrieve data through scoped tools and cite its sources.
3. **Stage 2—Transparent loop.** The system exposes state, tool calls, budgets, stop conditions and errors.
4. **Stage 3—Preparation autonomy.** The agent creates drafts, simulations and recommendations without external side effects.
5. **Stage 4—Gated action.** Consequential actions pause for verified approval; low-risk actions follow deterministic policy.
6. **Stage 5—Bounded autonomy.** The system acts within narrow, monitored authority and escalates exceptions.
7. **Stage 6—Institutional autonomy.** Multiple loops and agents operate with versioned skills, tool registries, durable execution, continuous evaluation, incident response and audit.

Each stage should have tests. Tool contracts need valid, invalid and unauthorized examples. Loop tests need simulated failures, partial results, conflicting evidence, delayed approval and restart recovery. Evaluation should measure not only final-answer quality but also action correctness, unnecessary tool calls, policy compliance, cost, latency, completion accuracy and recovery behavior.

17 Design Principles for the Agentic Age

The analysis can be condensed into ten design principles.

1. **Treat agency as a system property.**
2. **Separate proposal from execution authority.**
3. **Design tools around user outcomes, not raw API topology.**
4. **Keep skills and tools conceptually distinct but operationally coordinated.**
5. **Use MCP to standardize boundaries, not to conceal responsibility.**
6. **Make state transitions explicit and resumable.**
7. **Define stopping, waiting and escalation as first-class outcomes.**
8. **Concentrate human approval on consequential ambiguity.**
9. **Preserve evidence separately from mutable working memory.**
10. **Increase autonomy only as validation, observability and institutional confidence increase.**

These principles suggest a broader interpretation of autonomous intelligence. It is not a miniature person trapped inside a machine. It is a designed organization of reasoning, procedures, capabilities, controls and records. The anthropomorphic metaphor can help us understand roles, but engineering discipline requires us to see the interfaces.

18 Conclusion: The Heartbeat and the Chassis

The defining movement from generative AI to autonomous intelligence is not simply the addition of more powerful models. It is the construction of a loop around the model.

The loop receives a purpose, observes a state, asks the model what should happen next, evaluates the proposed action, invokes an admissible capability, receives the consequence, updates

memory and decides whether to continue. This cycle gives the system continuity. It makes the process responsive to evidence rather than fixed in advance. It allows an objective expressed in ordinary language to unfold into a sequence of computational acts.

Yet the loop does not operate alone. Plugins package coherent extensions. Skills contribute reusable methods. Tools provide executable hands. MCP supplies a common language for discovering and invoking capabilities. APIs connect the integration layer to concrete services. Internal and external resources provide the environment in which work occurs. Memory makes the process path-dependent. The orchestrator supplies the clock and lifecycle. Governance supplies the boundaries of legitimate action. Auditability preserves the institutional memory of what the system actually did.

The minimum logical agent therefore contains a goal, decision mechanism, action channel, feedback, state and stopping rule. The minimum responsible agent adds identity, permission, validation, oversight, operational limits and evidence. This distinction is not semantic. It separates an interesting demonstration from a system that can be trusted with human, financial or institutional consequences.

The most important sentence is consequently also the simplest:

“A model becomes operationally agentic when it can act, observe the result, update its state and decide what happens next—within explicit authority and with a reliable way to stop.”

The agentic loop is the heartbeat of autonomous intelligence. Governance is its chassis. Without the heartbeat, the model cannot sustain purposeful action. Without the chassis, its power cannot be safely carried into the world.

References

- Model Context Protocol. Architecture. MCP Specification, version 2025-11-25, 2025. URL <https://modelcontextprotocol.io/specification/2025-11-25/architecture/index>. Accessed 24 July 2026.
- Model Context Protocol. Architecture overview, 2026. URL <https://modelcontextprotocol.io/docs/learn/architecture>. Accessed 24 July 2026.
- National Institute of Standards and Technology. Artificial intelligence risk management framework (ai rmf 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023. URL <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>.
- National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, U.S. Department of Commerce, 2024. URL <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.
- OpenAI. Human-in-the-loop. OpenAI Agents SDK Documentation, 2026a. URL https://openai.github.io/openai-agents-python/human_in_the_loop/. Accessed 24 July 2026.
- OpenAI. Mcp server: How a plugin gets tools and access to external systems. OpenAI Developer Documentation, 2026b. URL <https://developers.openai.com/plugins/concepts/mcp-server>. Accessed 24 July 2026.
- OpenAI. Running agents: The agent loop. OpenAI Agents SDK Documentation, 2026c. URL https://openai.github.io/openai-agents-python/running_agents/. Accessed 24 July 2026.
- OpenAI. Define tools: Translate plugin use cases into a focused mcp tool surface. OpenAI Developer Documentation, 2026d. URL <https://developers.openai.com/plugins/plan/tools>. Accessed 24 July 2026.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://proceedings.mlr.press/v235/zhou24r.html>.

A A Minimal Agentic Loop in Pseudocode

```

state = initialize(goal, policy, budget, memory)

while True:
    context = assemble_context(state, available_tools)
    proposal = model.decide(context)

    if proposal.type == "final":
        assert completion_criteria(state, proposal)
        return finalize(proposal, state)

    if budget_exhausted(state):
        return escalate("Budget exhausted", state)

    decision = policy_gate(proposal, state)

    if decision.requires_human:
        state = persist_and_pause(state, decision)
        return request_approval(state)

    if decision.rejected:
        state = update(state, decision.as_observation())
        continue

    result = execute_once(
        decision.authorized_action,
        timeout=state.timeout,
        idempotency_key=state.next_key
    )

    state = update(state, proposal, decision, result)
    audit.append(state.last_transition)

    outcome = evaluate_stop_rule(state)
    if outcome in {"complete", "escalate", "fail_safely", "wait"}:
        return close_run(outcome, state)

```

B Minimum-Requirements Checklist

Requirement	Status	Evidence to retain
Goal and success criterion	☐	Versioned objective, scope, exclusions and owner
Trigger and lifecycle	☐	User, schedule or event; start and end timestamps
Model and instructions	☐	Model identifier, configuration and instruction version
Available skills	☐	Skill names, owners, versions and tests

Requirement	Status	Evidence to retain
Available tools	☐	Tool registry, schemas, permissions and side effects
State and memory	☐	Working-state schema, retrieval provenance and retention
Policy gate	☐	Identity, role, authorization and decision outcome
Human approval	☐	Approver, resolved target, decision and timestamp
Execution controls	☐	Timeout, retry, idempotency and recovery settings
Stopping conditions	☐	Complete, wait, escalate, budget and failure criteria
Audit trail	☐	Tool calls, results, transitions, errors and artifacts
Evaluation	☐	Scenario tests, policy tests and operational metrics